Study Notes: Days 8–14 — LINQ, Async, Memory

These notes follow Month 1, Week 2 of the roadmap. The examples use modern C#; each standalone console example is independent. EF Core examples are explicitly identified and require an EF Core application and a configured provider. Examples are illustrative, not claims about your projects.

Day 8 — LINQ: Where, Select, SelectMany, OrderBy, GroupBy, Join, Any/All/First/Single

Month 1 — Foundations, Week 2 — LINQ, Async, Memory.

Goal: Build readable queries, understand each operator's result and execution, and choose terminal operators that express the actual business rule.

`Where`: filtering

`Where` retains elements for which a predicate returns `true`. For an `IEnumerable<T>`, it normally creates a deferred iterator: the predicate runs as the consumer requests elements, not when the query variable is assigned. Filters can be combined or chained; put cheap, selective filters early when working in memory, provided it does not change semantics or obscure intent.

With an `IQueryable<T>`, the predicate is usually an expression tree passed to a provider. The provider may translate it to SQL rather than running it in C#. Inspect generated SQL and query plans when performance matters; the database optimizer decides its own execution strategy. Avoid using helper methods in a database predicate unless the provider can translate them.

`Select`: projection

`Select` maps each input element to one output element. It can select a property, construct a DTO, or calculate a value. Projection helps keep the result focused; in database queries, selecting only required columns can reduce data transfer and materialization cost. It does not automatically make a query fast: joins, filters, indexes, and the database plan still matter.

For in-memory sequences the selector is deferred, so expensive work or side effects may run again on re-enumeration. `Select` does not flatten nested collections: selecting `order.Items` gives a sequence of item sequences.

`SelectMany`: flattening

`SelectMany` maps each input to a sequence and flattens those sequences into one stream. For example, selecting items from each order produces one sequence of items rather than one sequence per order. An order with no items contributes no output. Use the result-selector overload if each flattened item must retain its parent order's information.

Flattening can multiply rows significantly, especially when translated into joins or cross joins. Filter early where appropriate and check whether duplicates are intended. It is not interchangeable with `Select` when the caller needs to preserve the grouping by parent.

`OrderBy` and `ThenBy`: sorting

`OrderBy` orders elements by a key; `OrderByDescending` reverses the direction. Use `ThenBy` or `ThenByDescending` for subsequent keys. A second `OrderBy` starts a new primary ordering rather than adding a secondary key.

For LINQ to Objects, sorting is deferred but must consume and buffer the input before yielding sorted results. Its comparison cost is generally O(n log n), with O(n) auxiliary storage. Database providers translate ordering where supported; when paging, specify a deterministic tie-breaker such as a unique ID, otherwise rows tied on the sort key can move between pages.

`GroupBy`: grouping

`GroupBy` associates each key with its matching elements, producing `IGrouping<TKey, TElement>` values in LINQ to Objects. It is useful for summaries such as total order value by customer. In-memory grouping is deferred but buffers source elements when enumeration begins, so large groups can consume substantial memory.

SQL supports aggregate grouping well, but an arbitrary .NET grouping object is not a SQL result type. EF Core translation depends on the query and provider: a `GroupBy` followed by supported aggregate projections often translates, while other shapes may require client-side grouping or fail to translate. Inspect the SQL and volume rather than assuming either outcome.

`Join`: matching keys

`Join` is an inner equijoin: it matches outer and inner elements by selected keys and emits a result for each match. Unmatched elements are omitted; duplicate keys can generate multiple output rows. LINQ to Objects uses a lookup of the inner sequence, so it buffers that side and usually takes O(n + m + r) time and O(m) additional space, where `n` and `m` are input sizes and `r` is the output size.

Use an inner join when unmatched rows should disappear. A left join needs a different query shape, typically `GroupJoin` with `DefaultIfEmpty` in widely used LINQ patterns; support and resulting SQL depend on the provider. Ensure nullable keys, equality rules, and duplicate matches reflect the business requirement.

`Any`, `All`, `First`, and `Single`

`Any` tests whether at least one element exists or satisfies a predicate; `All` tests whether every element satisfies one. They can stop once the answer is known. Importantly, `All` returns `true` for an empty sequence, while `Any` returns `false`. A provider may translate existence checks to an efficient `EXISTS`, but inspect the actual query.

`First` returns the first matching element and throws if there is none; `FirstOrDefault` returns the type's default instead. If ordering matters, define it explicitly—especially in SQL, where unspecified row order is not reliable. `Single` returns the only matching element and throws if there are zero or multiple matches; `SingleOrDefault` permits zero but still throws for multiple matches. Use `Single` when uniqueness is an invariant, not as a substitute for a database unique constraint under concurrent writes. For value types, an `OrDefault` result can be indistinguishable from an actual default-valued element unless the API handles absence explicitly.

Code example

```
using System;
using System.Linq;

var customers = new[]
{
    new Customer(1, "Ada"),
    new Customer(2, "Grace"),
    new Customer(3, "Lin")
};
var orders = new[]
{
    new Order(101, 1, new[] { 20m, 30m }),
    new Order(102, 1, new[] { 10m }),
    new Order(103, 2, Array.Empty<decimal>())
};

var totals = orders
    .Where(o => o.Items.Length > 0)
    .Select(o => new { o.Id, Total = o.Items.Sum() })
    .OrderByDescending(o => o.Total)
    .ThenBy(o => o.Id);
foreach (var order in totals)
    Console.WriteLine($"{order.Id}: {order.Total}"); // 101: 50; 102: 10

var items = orders.SelectMany(o => o.Items);
Console.WriteLine(items.Count()); // 3

var grouped = orders.GroupBy(o => o.CustomerId)
    .Select(g => new { CustomerId = g.Key, Count = g.Count() });
foreach (var group in grouped)
    Console.WriteLine($"{group.CustomerId}: {group.Count}"); // 1: 2; 2: 1

var matches = customers.Join(orders, c => c.Id, o => o.CustomerId,
    (c, o) => $"{c.Name}: order {o.Id}");
Console.WriteLine(string.Join("; ", matches)); // Lin has no match

Console.WriteLine(orders.Any(o => o.Items.Length == 0)); // True
Console.WriteLine(Array.Empty<int>().All(n => n > 0));   // True
Console.WriteLine(orders.First(o => o.CustomerId == 1).Id); // 101
Console.WriteLine(orders.Single(o => o.Id == 102).Id);      // 102

record Customer(int Id, string Name);
record Order(int Id, int CustomerId, decimal[] Items);
```

This example uses LINQ to Objects. It does not demonstrate which expressions a database provider can translate.

Likely interview questions

1. What is the difference between `Select` and `SelectMany`?
2. What happens when you use a second `OrderBy` instead of `ThenBy`?

3. Why might `GroupBy` consume substantial memory even though it is deferred?
4. What do duplicate keys and unmatched rows do in an inner join?
5. What does `All` return for an empty input, and why is that relevant?
6. When would you use `Single` rather than `First`, and what guarantees uniqueness under concurrent database writes?
7. Why is `OrderBy` important before `First` or pagination in a database query?
8. How would you verify whether an EF Core grouping or join is translated efficiently?

Common pitfalls / misconceptions

- Assuming every operator streams element by element: sorting and in-memory grouping buffer input.
- Using `FirstOrDefault` where a missing result and a default-valued result have different meanings.
- Treating `Single` as a database uniqueness guarantee or assuming SQL has an implicit row order.
- Assuming a LINQ-to-Objects expression is necessarily translatable by a database provider.

Cheat-sheet recap

- `Where` filters; `Select` maps; `SelectMany` flattens.
- `OrderBy` sets the primary ordering; `ThenBy` adds tie-breakers.
- `GroupBy` groups; `Join` matches keys and can multiply rows.
- `Any` means at least one; `All` is true on empty input.
- `First` expresses first match; `Single` asserts exactly one match.

Day 9 — IEnumerable vs IQueryable — important for your background

Month 1 — Foundations, Week 2 — LINQ, Async, Memory.

Goal: Distinguish in-process iteration from provider-translated queries and recognize where a query crosses that boundary.

`IEnumerable<T>`: iteration and delegates

`IEnumerable<T>` represents something that can be iterated. LINQ operators from `Enumerable` generally receive compiled delegates such as `Func<T, bool>` and run their logic in the process as elements are requested. The source could already be a list, a generated iterator, a file stream, or a database query; the interface alone does not promise that elements are in memory, that enumeration is cheap, or that it can be repeated safely.

It is a good contract for a method that needs only iteration. If a method needs a known count or indexed access, a more specific contract may communicate that requirement better. Do not expose a deferred sequence backed by a disposed database context or other resource.

`IQueryable<T>`: expression trees and providers

`IQueryable<T>` extends `IEnumerable<T>` and adds an expression tree and a query provider. Operators from `Queryable` normally receive expression trees rather than ordinary delegates. A provider such as EF Core inspects the tree, translates supported operations to its query language, and executes the request when the result is consumed or a terminal operator runs.

`IQueryable<T>` is not inherently SQL: `AsQueryable()` on a list generally creates an in-memory LINQ-to-Objects query provider. Conversely, an EF query may execute SQL when enumerated through `IEnumerable<T>`. The important question is where each operation is applied, which provider owns it, and when execution happens.

The boundary: filtering, materialization, and translation

In an EF Core query, applying `Where`, projection, and a limit before materialization normally lets the provider send those operations to the database. `ToList` or `ToListAsync` executes the query and produces a list. Calling `AsEnumerable()` usually changes which subsequent extension methods are selected without immediately executing the query: operators after that boundary use `Enumerable` and run in the application as rows are enumerated. The provider still executes the earlier portion.

Moving a selective filter after `AsEnumerable()` can transfer many unnecessary rows and use application CPU and memory. It may be justified when a required operation cannot be translated and the database-side portion has already bounded the data. In modern EF Core, an untranslatable expression in most query positions normally throws rather than silently filtering all rows on the client; limited client evaluation in the top-level projection is supported. Provider and version details matter.

Use `ToQueryString()` or EF logging to inspect SQL, measure database and network costs, and examine the actual execution plan and indexes. Returning `IQueryable<T>` across application boundaries grants callers flexibility but leaks provider semantics, lifetime requirements, and performance control. A focused query method returning a materialized DTO can be safer; streaming can be appropriate for large results if resource lifetime and cancellation are explicit.

Code examples

This standalone example shows that `IQueryable` need not involve a database:

```
using System;
using System.Collections.Generic;
using System.Linq;

var values = new List<int> { 1, 2, 3, 4 };
IQueryable<int> query = values.AsQueryable().Where(n => n > 2);
IEnumerable<int> local = query.AsEnumerable().Where(n => n % 2 == 0);

Console.WriteLine(query.Expression);     // expression tree, not SQL
Console.WriteLine(string.Join(", ", local)); // 4
values.Add(6);
Console.WriteLine(string.Join(", ", local)); // 4, 6: evaluated again
```

In an EF Core application, assuming `db.Orders` is a `DbSet<Order>` and `Order` has `Id`, `Status`, and `Total`:

```
// Requires a live DbContext and Microsoft.EntityFrameworkCore.
var query = db.Orders
    .Where(o => o.Status == "Open")
    .OrderBy(o => o.Id)
```

```
    .Select(o => new { o.Id, o.Total })
    .Take(100);

Console.WriteLine(query.ToQueryString()); // inspect provider-generated SQL
var rows = await query.ToListAsync(cancellationToken);
```

The projection contains no entity instance, so this result does not track `Order` entities. SQL translation, collation, indexes, and exact syntax depend on the provider and database configuration.

Likely interview questions

1. How do `Enumerable.Where` and `Queryable.Where` differ in the kind of predicate they accept?
2. Does `IEnumerable<T>` necessarily mean that all records are already in memory?
3. What changes when `AsEnumerable()` appears before a filter, and does it execute the query immediately?
4. Can `IQueryable<T>` execute without a database?
5. What happens when EF Core cannot translate a predicate?
6. How would you investigate an endpoint that retrieves too many rows?
7. What are the tradeoffs of returning `IQueryable<T>` from a repository or service?
8. How do context lifetime and repeated enumeration affect a deferred EF query?

Common pitfalls / misconceptions

- Calling `AsQueryable()` on an existing list and expecting database execution.
- Materializing or crossing to `Enumerable` before a selective filter without recognizing the data-transfer cost.
- Claiming all untranslatable EF Core expressions are silently evaluated on the client.
- Treating a provider-backed query as independent of its context lifetime.

Cheat-sheet recap

- `IEnumerable`: iterate; `Enumerable` operators execute delegates in process.
- `IQueryable`: expression tree plus provider; translation depends on that provider.
- `AsEnumerable` changes subsequent operator binding; `ToList` materializes.
- Inspect SQL, row counts, execution plans, and the point where work changes location.

Day 10 — Deferred execution

Month 1 — Foundations, Week 2 — LINQ, Async, Memory.

Goal: Predict when a query runs, what it observes, and how its evaluation affects resources, correctness, and performance.

Construction versus execution

Many sequence-returning LINQ operators describe a pipeline when called and perform their work only on enumeration. For example, `Where` and `Select` usually process each requested element on demand. A `foreach`, `ToList`, or scalar terminal operator such as `Count` or `Any` triggers enumeration. Query construction

and execution are separate moments; exceptions thrown by a predicate or source can therefore occur later than the line that created the query.

Deferred does not necessarily mean streaming. `OrderBy` must read and sort its source before yielding the first ordered result, and LINQ-to-Objects `GroupBy` builds groups when enumeration begins. By contrast, `Where` can usually yield matches incrementally and `Take` can stop early. Exact behavior depends on the operator and provider.

Re-enumeration, mutation, and captured variables

Enumerating a deferred query twice can redo filtering, calculations, network calls, or database queries; it can also produce different results if the source or a captured variable changed. A lambda captures a variable, not a frozen copy of its value. This is useful for some scenarios but dangerous when code assumes the query has already captured a threshold or result set.

Materialize with `ToList` or `ToArray` when a stable collection of results, repeated access, or a clear resource boundary is needed. This is a **shallow** snapshot: referenced objects can still change, and materializing a database query is not automatically a transactionally consistent snapshot across multiple separate queries. The cost is upfront work and memory. For a one-pass, large sequence, streaming may be preferable.

Mutating a `List<T>` during active enumeration typically invalidates its enumerator and causes an exception. Modifying the list before a new enumeration, as in the example below, is different. A deferred query using a disposed context or stream may fail when enumerated; keep the resource alive through enumeration or materialize within its lifetime.

Code example

```
using System;
using System.Collections.Generic;
using System.Linq;

var numbers = new List<int> { 1, 2, 3 };
int minimum = 2;
var query = numbers.Where(n => n >= minimum);

Console.WriteLine(string.Join(", ", query)); // 2, 3
minimum = 3;
numbers.Add(4);                              // no enumeration is active
Console.WriteLine(string.Join(", ", query)); // 3, 4

var snapshot = query.ToList();               // evaluates now
numbers.Add(5);
minimum = 1;
Console.WriteLine(string.Join(", ", snapshot)); // 3, 4
Console.WriteLine(string.Join(", ", query));    // 1, 2, 3, 4, 5
```

The snapshot contains `int` values. If it instead contained mutable objects, it would hold references to the same objects, not deep copies.

Likely interview questions

1. When does a deferred LINQ predicate actually execute?
2. Why can `Count()` followed by `foreach` be problematic on the same query?
3. What is the difference between deferred streaming and deferred buffering?
4. What does a captured variable contribute when it changes between query creation and enumeration?
5. What does `ToList` stabilize, and what does it not copy?
6. What happens if a query outlives its `DbContext`?
7. When would you prefer streaming to materialization?
8. How would you detect repeated database execution in an endpoint?

Common pitfalls / misconceptions

- Assuming query declaration executes the query or freezes captured values.
- Enumerating an expensive or single-use source repeatedly.
- Assuming a list snapshot deep-copies mutable objects.
- Disposing the resource backing a sequence before consuming it.

Cheat-sheet recap

- Deferred means work starts on consumption, not necessarily that results stream.
- Re-enumeration can repeat work and observe changed state.
- Materialize deliberately; balance stability and lifetime against memory and latency.

Day 11 — async/await: Task, Task, async, await

Month 1 — Foundations, Week 2 — LINQ, Async, Memory.

Goal: Explain how asynchronous methods represent and await work, and use them without blocking threads or losing failures.

`Task` and `Task<T>`

`Task` represents an operation that may be incomplete, successfully completed, canceled, or faulted. `Task<T>` also carries a result of type `T`. They represent completion, not necessarily a dedicated thread: an asynchronous network read can wait for I/O without occupying a thread for the entire wait. A task may already be completed when returned.

Prefer `Task` or `Task<T>` for asynchronous APIs that callers need to await. Return `Task` when there is no result and `Task<T>` when there is. A `CancellationToken` communicates a request to stop; it is cooperative and must be passed to operations that honor it. Cancellation is not rollback: a remote service may already have processed a request.

`async` and `await` mechanics

`async` allows a method to use `await`. The method starts synchronously and continues until it completes or reaches an incomplete await. The compiler transforms it into a state machine that preserves the required state across suspension. At an incomplete await, control returns to the caller; after the awaited operation completes, the continuation resumes according to the captured context or scheduler where applicable. If the task is already complete, execution may continue synchronously without suspension.

`await` retrieves the result or propagates the operation's failure or cancellation. It does not start every task automatically: many APIs return tasks representing work already started; the behavior depends on the API. It does not by itself create a thread. Asynchronous I/O improves scalability by freeing threads while waiting; CPU-bound work still consumes CPU while it runs.

Context, failures, and design tradeoffs

By default, an await may capture a `SynchronizationContext` or a non-default task scheduler. UI code often requires a return to the UI context for UI updates. ASP.NET Core normally has no custom request `SynchronizationContext`, unlike classic ASP.NET; do not assume continuations resume on the same thread. `ConfigureAwait(false)` avoids requesting the captured context for that await; it is useful in context-independent library code but does not guarantee a particular thread or make non-thread-safe state safe.

Avoid `.Result`, `.Wait()`, and sync-over-async on request or UI threads. Blocking can waste thread-pool threads and, in a context that requires the blocked thread to run the continuation, deadlock. Await through the call chain instead. Use `async void` primarily for event handlers that require it: callers cannot await it as a task or reliably observe its exception through a returned task. Background operations need an explicit owner, lifetime, shutdown behavior, and error handling rather than an unobserved fire-and-forget call.

Code example

```
using System;
using System.Threading;
using System.Threading.Tasks;

using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(2));
try
{
    int value = await GetValueAsync(cts.Token);
    Console.WriteLine(value); // 42 when the delay completes
}
catch (OperationCanceledException) when (cts.IsCancellationRequested)
{
    Console.WriteLine("The operation was canceled.");
}

static async Task<int> GetValueAsync(CancellationToken cancellationToken)
{
    await Task.Delay(TimeSpan.FromMilliseconds(50), cancellationToken);
    return 42;
}
```

`Task.Delay` simulates waiting; a real API should pass the token to its actual asynchronous I/O operation. Unexpected errors are not swallowed by the example.

Likely interview questions

1. What does a `Task` represent, and does it always represent a thread?
2. What happens before and after the first incomplete `await`?

3. How does `Task<T>` differ from `Task`?
4. How are exceptions and cancellation observed by a caller?
5. Why can `.Result` deadlock in a UI application, and what is different in ASP.NET Core?
6. What does `ConfigureAwait(false)` actually change?
7. Why is `async void` usually unsuitable for application APIs?
8. Does cancellation prove that a remote side effect did not happen?

Common pitfalls / misconceptions

- Saying `async` creates a thread or makes CPU work automatically parallel.
- Blocking on an async operation and assuming the only possible issue is a deadlock; thread-pool starvation also matters.
- Losing failures by launching a task with no owner or observable completion.
- Assuming cancellation guarantees rollback or that continuations return to the original thread in ASP.NET Core.

Cheat-sheet recap

- `Task` represents completion; `Task<T>` also represents a result.
- `await` suspends an async method only when needed and observes completion.
- Pass cancellation through, await the full chain, and handle errors at an appropriate boundary.
- Context capture varies by environment; avoid sync-over-async.

Day 12 — Concurrency: Thread, ThreadPool, Task, Parallel, Task.WhenAll

Month 1 — Foundations, Week 2 — LINQ, Async, Memory.

Goal: Choose the right concurrency mechanism for the workload, understand failure behavior, and bound resource use.

Concurrency and parallelism

Concurrency means multiple operations make progress during overlapping periods; parallelism means work executes simultaneously. Overlapping network requests can improve latency without dedicating a thread to each waiting operation. CPU-bound parallel work may run on multiple cores, but competes for finite CPU and can make a loaded server less responsive if overused.

`Thread` and `ThreadPool`

`Thread` represents an OS-backed managed thread with its own execution stack. A dedicated thread can be appropriate for a long-running loop with special lifetime or thread-affinity requirements, but creating many threads has scheduling and memory overhead. Application code usually should not manually create one thread per request or small work item.

The .NET thread pool reuses workers for short-lived work and supports task scheduling. Blocking workers on long waits reduces available capacity and may lead to thread-pool starvation under load; the pool can add workers, but ramp-up and resource limits still affect latency. Asynchronous I/O avoids occupying a worker for the whole wait. Thread-pool workers are not a durable job queue: process termination can interrupt work.

`Task` and `Task.Run`

`Task` models an operation's completion; it may represent CPU work, async I/O, or some other completion source. `Task.Run` queues a delegate to the thread pool and is useful for offloading CPU-bound work from a UI thread. In a server request, indiscriminately wrapping CPU work in `Task.Run` adds scheduling overhead and does not create extra CPU capacity; use it only when the scheduling tradeoff is deliberate.

Do not wrap naturally asynchronous I/O in `Task.Run` merely to make it asynchronous. Use the I/O API's async method directly. A request that starts background work without awaiting it needs an explicitly managed background service or queue if it must survive beyond the request; an in-process queue still is not durable by itself.

`Parallel`

`Parallel.For` and `Parallel.ForEach` are primarily for independent synchronous CPU-bound iterations. They partition work and use the thread pool; tune `MaxDegreeOfParallelism` when necessary and measure. Shared mutable state requires synchronization, and per-item scheduling can make small workloads slower than a simple loop.

Do not pass an `async` lambda to synchronous `Parallel.ForEach`: it becomes `async void`, so the loop cannot correctly await its completion or propagate failures as intended. For asynchronous bodies, modern .NET provides `Parallel.ForEachAsync`, which returns a task and supports a degree-of-parallelism limit and cancellation. The limit should reflect downstream capacity, not just CPU count.

`Task.WhenAll`

`Task.WhenAll` creates a task that completes after all supplied tasks complete. It does not itself start tasks: create or invoke the operations first. For `Task<T>` inputs, the resulting array corresponds to input order, not completion order. If any input faults, the combined task faults; if none faults but at least one is canceled, it is canceled. Awaiting it propagates an exception, while the combined task's `Exception` contains the aggregate fault information. Other operations continue unless cancellation is explicitly coordinated.

Launching thousands of remote calls at once can exhaust connection pools, hit rate limits, or overload the dependency. Bound concurrency and consider timeouts, cancellation, and retry safety. EF Core `DbContext` is not thread-safe: do not run parallel operations on the same instance; use separate contexts where parallel database work is justified.

Code example

```
using System;
using System.Threading;
using System.Threading.Tasks;

using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(2));
Task<int> first = ReadAsync(1, cts.Token);
Task<int> second = ReadAsync(2, cts.Token);
Task<int[]> combined = Task.WhenAll(first, second);

try
{
    int[] values = await combined;
    Console.WriteLine(string.Join(", ", values)); // 10, 20 in input order
```

```
}
catch (OperationCanceledException) when (cts.IsCancellationRequested)
{
    Console.WriteLine("The operation was canceled.");
}
catch (Exception) when (combined.IsFaulted)
{
    foreach (Exception error in combined.Exception!.Flatten().InnerExceptions)
        Console.WriteLine(error.Message);
    throw; // preserve failure for the caller
}

static async Task<int> ReadAsync(int id, CancellationToken cancellationToken)
{
    await Task.Delay(50, cancellationToken);
    return id * 10;
}
```

This deliberately runs only two operations. In a high-cardinality workload, use a bounded approach rather than starting an unbounded task per item.

Likely interview questions

1. How is concurrency different from parallelism?
2. When is a dedicated `Thread` justified over a thread-pool task?
3. What happens to a server when many thread-pool workers block?
4. When is `Task.Run` useful, and why is it generally unnecessary around async I/O?
5. Why is an async lambda unsafe with synchronous `Parallel.ForEach`?
6. Does `Task.WhenAll` start tasks, and in what order are results returned?
7. How do you observe multiple failures from `WhenAll`?
8. How would you bound fan-out to a rate-limited service, and can you share one `DbContext` across calls?

Common pitfalls / misconceptions

- Equating a task with one dedicated thread or assuming every concurrent operation is parallel.
- Offloading already-asynchronous I/O with `Task.Run`.
- Using `Parallel.ForEach` with an async delegate or ignoring its returned work.
- Assuming `WhenAll` cancels siblings on the first failure or limits concurrency.

Cheat-sheet recap

- Thread = execution resource; thread pool = reused workers; task = completion abstraction.
- Use direct async I/O for waits; use parallel CPU work only when measured and bounded.
- `WhenAll` joins existing tasks and preserves result input order.
- Manage fan-out, errors, cancellation, and dependency capacity explicitly.

Day 13 — Synchronization: lock, SemaphoreSlim, Mutex, race conditions, deadlocks

Month 1 — Foundations, Week 2 — LINQ, Async, Memory.

Goal: Protect shared state with the correct primitive and explain the resulting correctness, scope, and performance tradeoffs.

Race conditions

A race condition occurs when correctness depends on an uncontrolled interleaving of concurrent operations. For example, `count++` is a read-modify-write sequence: two callers can read the same value and overwrite each other's updates. An individual read or write being atomic does not make the entire compound operation atomic. Use `Interlocked.Increment` for a simple shared counter, a lock for a larger invariant, or avoid sharing mutable state where possible.

The right scope matters. An in-process lock protects cooperating code in that process; it cannot protect an invariant across multiple application instances or direct database writers. For persisted state, use database constraints, transactions, appropriate isolation, or optimistic concurrency as the situation requires. Avoid holding a process lock over remote calls or long computations because it serializes work and increases latency.

`lock`

`lock` provides mutual exclusion around a synchronous critical section using a shared lock object. A `lock` statement releases the lock even if the body throws. All access participating in the invariant must use the same synchronization strategy. Keep the lock object private and dedicated: locking on public objects, strings, or `this` can allow unrelated code to interfere.

The underlying monitor is thread-affine and supports reentrancy by the owning thread. Do not `await` inside a C# `lock` body; the compiler rejects it. For async coordination use an async-compatible primitive instead. Locking has contention and scheduling costs, so keep critical sections short and measure rather than assuming synchronization is free.

`SemaphoreSlim`

`SemaphoreSlim` controls access to a limited number of slots. A count of one can protect an async critical section; a larger count can limit concurrency, such as calls to a downstream service. `WaitAsync` avoids blocking a worker while waiting and accepts a cancellation token. Once acquisition succeeds, release in `finally`; if waiting was canceled before acquisition, do not release a slot that was never acquired.

Unlike a monitor, a semaphore does not enforce thread ownership, so correct pairing is the application's responsibility. It is an in-process primitive, not a distributed lock, and fairness is not a contractual guarantee to build correctness around. Dispose it only after users are finished.

`Mutex`

A `Mutex` provides exclusive, thread-owned synchronization and can be named for coordination between processes on the same machine, subject to OS naming, permissions, and platform behavior. It is typically heavier than an in-process lock; do not use it as a distributed lock across machines. The acquiring thread must release it, so it is generally unsuitable for holding across an `await`, where continuation may resume on a different thread.

An abandoned mutex indicates a prior owner terminated without releasing it; the acquiring thread receives `AbandonedMutexException` and gains ownership. Treat the protected state as potentially inconsistent and

repair or reject it appropriately, then release ownership. A process crash and external side effects require a recovery design, not just a mutex.

Deadlocks and prevention

A deadlock is a cycle of waits in which each participant needs a resource another participant will not release. One common example is thread A holding lock 1 and waiting for lock 2 while thread B holds lock 2 and waits for lock 1. Another is blocking a context thread on a task whose continuation needs that same context. Async code can deadlock too; `await` prevents thread blocking but does not eliminate circular resource waits.

Prevent deadlocks by acquiring multiple locks in a consistent global order, minimizing nested locks, avoiding callbacks or I/O while holding locks, and awaiting async work instead of blocking context threads. Timeouts and cancellation can limit waiting and aid diagnosis but do not by themselves establish correctness or prevent every deadlock. Investigate with thread/task dumps, traces, and the actual lock ownership and wait graph.

Code example

```
using System;
using System.Threading;
using System.Threading.Tasks;

var counter = new Counter();
var tasks = new Task[100];
for (int i = 0; i < tasks.Length; i++)
    tasks[i] = Task.Run(counter.Increment);
await Task.WhenAll(tasks);
Console.WriteLine(counter.Value); // 100

using var gate = new SemaphoreSlim(2, 2);
await Task.WhenAll(UseSlotAsync(gate, 1), UseSlotAsync(gate, 2),
    UseSlotAsync(gate, 3));

static async Task UseSlotAsync(SemaphoreSlim gate, int id)
{
    await gate.WaitAsync(); // acquisition succeeded before entering try
    try
    {
        await Task.Delay(20); // at most two bodies hold a slot at once
        Console.WriteLine($"Completed {id}");
    }
    finally
    {
        gate.Release();
    }
}

sealed class Counter
{
    private readonly object sync = new();
    private int value;
    public void Increment() { lock (sync) { value++; } }
```

```
    public int Value { get { lock (sync) { return value; } } }
}
```

Output order for the slot users is not guaranteed. A production method can pass a `CancellationToken` to `WaitAsync` and to its work; the `try/finally` should still start only after successful acquisition.

Likely interview questions

1. Why is `count++` unsafe under concurrent updates?
2. When would `Interlocked`, `lock`, or `SemaphoreSlim` be a better fit?
3. Why can you not `await` in a `lock` body?
4. How do you ensure a canceled semaphore wait does not cause an extra release?
5. How does a named mutex differ from a process-local lock, and what happens if it is abandoned?
6. Give a deadlock scenario and a concrete prevention strategy.
7. Why does an in-process lock not protect an invariant across multiple service instances?
8. How would you investigate lock contention or starvation in production?

Common pitfalls / misconceptions

- Protecting only writes while reading the same invariant without appropriate synchronization.
- Assuming an in-memory lock coordinates multiple processes or machines.
- Releasing a semaphore after a failed acquisition or forgetting to release after success.
- Assuming async code cannot deadlock, or holding a thread-owned mutex across `await`.

Cheat-sheet recap

- Race = uncontrolled interleaving changes correctness; protect the full invariant.
- `lock` is synchronous mutual exclusion; `SemaphoreSlim` supports async waiting and bounded slots.
- A named `Mutex` can coordinate local processes but is thread-owned and requires recovery for abandonment.
- Avoid circular waits; use consistent lock order and short critical sections.

Day 14 — Review + coding/interview questions

Month 1 — Foundations, Week 2 — LINQ, Async, Memory.

Goal: Connect LINQ evaluation, asynchronous execution, concurrency limits, and synchronization to realistic coding and production scenarios.

Review: how the topics connect

A query can be constructed now and executed later. Determine whether it is an in-memory delegate pipeline or an expression interpreted by a provider, where filtering occurs, which operators buffer, and whether re-enumerating repeats I/O. Prefer explicit projections and deterministic ordering for database results; inspect generated SQL rather than guessing.

An async method represents pending work with a task. Awaiting I/O does not dedicate a thread for the whole wait. Independent operations can overlap via `Task.WhenAll`, but unbounded fan-out can overwhelm a service, and a fault does not automatically cancel sibling tasks. Shared mutable state introduces races even in

async code; choose `lock`, `Interlocked`, or an async-compatible semaphore for the scope of the invariant. For cross-instance persisted state, use the database or another appropriately scoped coordination mechanism.

Coding exercise 1: LINQ review

Problem: Given orders with a customer ID, status, and total, find the total amount of open orders per customer. Return results sorted by total descending, then customer ID ascending. Include only customers with at least one open order.

Approach: Filter before grouping, aggregate each group, then specify both sort keys. The following is LINQ to Objects. For EF Core, verify that the provider translates the aggregate and inspect the SQL.

```
using System;
using System.Linq;

var orders = new[]
{
    new Order(2, "Open", 20m),
    new Order(1, "Open", 30m),
    new Order(1, "Closed", 999m),
    new Order(2, "Open", 10m),
    new Order(3, "Open", 50m)
};

var totals = orders
    .Where(o => o.Status == "Open")
    .GroupBy(o => o.CustomerId)
    .Select(g => new { CustomerId = g.Key, Total = g.Sum(o => o.Total) })
    .OrderByDescending(x => x.Total)
    .ThenBy(x => x.CustomerId)
    .ToList();

foreach (var item in totals)
    Console.WriteLine($"{item.CustomerId}: {item.Total}");
// 3: 50
// 1: 30
// 2: 30

record Order(int CustomerId, string Status, decimal Total);
```

For `n` input orders and `g` groups, expected time is O(n + g log g) with ordinary hash-based grouping and sorting, and additional space is O(n + g). Empty input produces an empty list. Decimal summation can overflow; negative totals and case sensitivity should follow the actual business rules.

Coding exercise 2: bounded async work

Problem: Process an input sequence asynchronously while allowing at most two operations inside the protected work at once. Wait for every operation and propagate failures.

Approach: Start tasks and use a `SemaphoreSlim` to bound the active work. Await `WhenAll`, and release only after successful acquisition. This example creates one task per item, so for very large or unbounded inputs prefer a bounded worker pipeline or `Parallel.ForEachAsync` rather than allocating all the waiting tasks.

```
using System;
using System.Linq;
using System.Threading;
using System.Threading.Tasks;

using var cancellation = new CancellationTokenSource();
using var gate = new SemaphoreSlim(2, 2);
int[] ids = { 1, 2, 3, 4 };

Task<int>[] tasks = ids.Select(id => ProcessBoundedAsync(
    id, gate, cancellation.Token)).ToArray();
int[] results = await Task.WhenAll(tasks);
Console.WriteLine(string.Join(", ", results)); // 10, 20, 30, 40

static async Task<int> ProcessBoundedAsync(
    int id, SemaphoreSlim gate, CancellationToken cancellationToken)
{
    await gate.WaitAsync(cancellationToken);
    try
    {
        await Task.Delay(20, cancellationToken); // stand-in for async I/O
        return id * 10;
    }
    finally
    {
        gate.Release();
    }
}
```

Result order follows input order, not completion order. Empty input completes with an empty result. A canceled wait does not release a slot; a fault or cancellation after acquisition still releases it. `WhenAll` waits for all supplied tasks and reports failure but does not automatically cancel the others.

Real-world tie-in: an actual async bug or race

Choose **one issue you personally encountered at TCS, Infor, or NCR**. Reflect on the actual symptom, affected workflow, competing operations or async wait, evidence from logs or reproduction, your contribution, the fix, and how the team verified it. If no such issue comes to mind, say so rather than presenting an invented incident. Omit confidential names and details.

Model answer shape: “In [actual system and workflow], we observed [specific symptom]. We reproduced or identified it using [real evidence]. The underlying issue was [actual interleaving or async behavior], which mattered because [impact]. I was responsible for [your contribution]. We considered [real alternatives] and chose [actual fix] because [tradeoff]. We verified it through [actual test or monitoring] and observed [verified result, or say it was not measured]. In hindsight, I would [specific improvement].”

Be precise about scope: an in-process semaphore may fix concurrency within one instance but not across replicas; an async API may free a thread but not make a shared dependency thread-safe. Mention these only if they actually relate to the incident.

Likely interview questions

1. How would you detect whether a filter executes in SQL or in application memory?
2. Why can a deferred query produce different results on its second enumeration?
3. Explain what happens when an async method awaits an incomplete task.
4. When would `Task.WhenAll` improve latency, and when might it overload a dependency?
5. How would you preserve all fault details from a group of tasks?
6. How would you protect a shared counter versus a longer async critical section?
7. Describe a deadlock and distinguish prevention from a timeout-based mitigation.
8. Walk through a real async or concurrency incident, including evidence and the scope of the fix.

Common pitfalls / misconceptions

- Focusing on syntax without explaining where execution occurs or what data crosses a boundary.
- Calling a query or task “async” and assuming it is automatically parallel, thread-safe, or bounded.
- Presenting a process-local lock as a cross-instance consistency solution.
- Inventing an incident, overstating personal ownership, or claiming an unmeasured improvement.

Cheat-sheet recap

- Identify **what executes, where, when, and how often** for each query.
- Await I/O, observe tasks and failures, and bound concurrent fan-out.
- Protect shared invariants with synchronization at the correct scope.
- For an incident: symptom → evidence → cause → action → verification → tradeoff.